

Tema 8.1 — Introducción a JavaFX y Scene Builder

0. Contexto del proyecto

Patrón MVC en JavaFX

JavaFX sigue el patrón **Modelo-Vista-Controlador (MVC)**:

Capa	Qué es	Archivo típico
Vista	La interfaz gráfica declarada en XML	*.fxml
Controlador	Gestiona los eventos y conecta vista con modelo	*Controller.java
Modelo	La lógica de negocio y los datos	Clases Java normales

Estructura de archivos del proyecto de referencia

```
src/  
  main/  
    java/org/example/demo/  
      HelloApplication.java <- Clase principal (extiende Application)  
      HelloController.java <- Controlador de la primera pantalla  
      SegundaPantallaController.java <- Controlador de la segunda pantalla  
      Contador.java <- Modelo: lógica del contador  
      Persona.java <- Modelo: datos de una persona  
      RepositorioPersonas.java <- Repositorio estático de personas  
    resources/org/example/demo/  
      hello-view.fxml <- Vista principal  
      segunda-pantalla.fxml <- Vista segunda pantalla
```

8.1.2 — Conociendo el entorno y primeros ejemplos

La vista FXML inicial

Un archivo `.fxml` define la interfaz de forma declarativa. El proyecto inicial tiene una vista con un **VBox**, una **Label** y un **Button**:

```
<VBox alignment="CENTER" spacing="20.0"
    xmlns:fx="http://javafx.com/fxml/1"
    xmlns="http://javafx.com/javafx/17.0.12"
    fx:controller="org.example.demo.HelloController">
    <padding>
        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
    </padding>

    <Label fx:id="welcomeText" />
    <Button onAction="#onHelloButtonClick" text="Hello!" textAlignment="CENTER" />
</VBox>
```

Propiedades clave del Button:

- `text` → texto visible en el botón.
- `textAlignment` → alineación del texto.
- `onAction` → método del controlador que se ejecuta al hacer clic (prefijo #).
- `fx:id` → identificador para referenciar el componente con @FXML.

Controlador básico — HelloController

```
import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class HelloController {

    @FXML
    private Label welcomeText;

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("Welcome to JavaFX Application!");
    }
}
```

- `@FXML` vincula la variable Java con el componente cuyo `fx:id` coincide.
- `setText()` modifica el texto de la `Label` en tiempo de ejecución.

Segunda forma de manejar eventos: setOnAction en initialize()

Se añade un segundo botón sin onAction en el FXML y se configura desde el controlador.

FXML actualizado:

```
<Label fx:id="welcomeText" />
<Button onAction="#onHelloButtonClick" text="Hello!" textAlignment="CENTER" />
<Button fx:id="boton" text="Pulsa!" textAlignment="CENTER" />
```

Controlador actualizado:

```
public class HelloController {

    @FXML private Label welcomeText;
    @FXML private Button boton;

    @FXML
    public void initialize() {
        boton.setOnAction(e -> System.out.println("Hola"));
    }

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("Welcome to JavaFX Application!");
    }
}
```

initialize():

- Se ejecuta automáticamente al cargar el FXML.
- Ideal para configurar eventos y valores iniciales.
- `setOnAction(e -> ...)` recibe una lambda que define el comportamiento al hacer clic.

Ejemplo: Contador que aumenta con cada pulsación

FXML:

```
<Label fx:id="welcomeText" />
<Button onAction="#onHelloButtonClick" text="Hello!" textAlignment="CENTER" />
<Button fx:id="contador" text="Contar" textAlignment="CENTER" />
<Label fx:id="contando" />
```

Controlador (versión con variable estática — enfoque inicial):

```
public class HelloController {

    static int cont;

    @FXML private Label welcomeText;
    @FXML private Button contador;
    @FXML private Label contando;

    @FXML
    public void initialize() {
        contador.setOnAction(e -> contando.setText(Integer.toString(++cont)));
    }

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("Welcome to JavaFX Application!");
    }
}
```

- `Integer.toString(++cont)` incrementa y convierte a `String` para `setText()`.
 - La `Label` `contando` muestra el valor actualizado en la ventana.
-

8.1.3 — Objetos (Modelo)

El **Modelo** representa la lógica de negocio. En lugar de una variable estática, se crea una clase independiente.

Clase Contador (Modelo)

```
public class Contador {  
  
    private int contador;  
  
    public Contador() { }  
  
    public void contar() {  
        contador++;  
    }  
  
    public int getContador() {  
        return contador;  
    }  
}
```

Controlador integrado con el Modelo

```
public class HelloController {  
  
    @FXML private Label welcomeText;  
    @FXML private Button contador;  
    @FXML private Label contando;  
  
    @FXML  
    public void initialize() {  
        Contador cont = new Contador();  
        contador.setOnAction(e -> {  
            cont.contar();  
            contando.setText(Integer.toString(cont.getContador()));  
        });  
    }  
  
    @FXML  
    protected void onHelloButtonClick() {  
        welcomeText.setText("Welcome to JavaFX Application!");  
    }  
}
```

Diferencia clave:

- Ya no hay `static int cont` en el controlador.
 - El objeto `Contador` encapsula la lógica; el controlador solo llama a sus métodos.
 - Respeta la separación MVC: la lógica vive en el Modelo, no en el Controlador.
-

8.1.4 — Moverse entre pantallas (Scene)

Para navegar entre pantallas se necesitan varias vistas FXML y sus controladores correspondientes.

- `hello-view.fxml` + `HelloController.java` → Pantalla 1
- `segunda-pantalla.fxml` + `SegundaPantallaController.java` → Pantalla 2

HelloApplication con escena estática

La Scene se saca como campo estático para cambiar su raíz desde cualquier controlador:

```
public class HelloApplication extends Application {

    private static Scene scene;

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(
            HelloApplication.class.getResource("hello-view.fxml")
        );
        scene = new Scene(fxmlLoader.load(), 320, 240);
        stage.setTitle("Hello!");
        stage.setScene(scene);
        stage.show();
    }

    private static Parent loadFXML(String fxml) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(
            HelloApplication.class.getResource(fxml + ".fxml")
        );
        return fxmlLoader.load();
    }

    static void setRoot(String fxml) throws IOException {
        scene.setRoot(loadFXML(fxml));
    }

    public static void main(String[] args) { launch(); }
}
```

- `scene` es `static` → accesible desde los controladores.
- `loadFXML(String)` → carga cualquier FXML por nombre (sin extensión).
- `setRoot(String)` → cambia el contenido de la ventana sin abrir una nueva.

FXML Pantalla 1 — hello-view.fxml

```
<VBox alignment="CENTER" spacing="20.0" ...
    fx:controller="org.example.demo.HelloController">
    <padding><Insets bottom="20.0" left="20.0" right="20.0" top="20.0" /></padding>

    <Label fx:id="welcomeText" />
    <Button onAction="#onHelloButtonClick" text="Hello!" textAlignment="CENTER" />
    <Button fx:id="contador" text="Contar" textAlignment="CENTER" />
    <Label fx:id="contando" />
    <Button text="Ir a pantalla 2" textAlignment="CENTER" onAction="#irAPantalla2" />
</VBox>
```

FXML Pantalla 2 — segunda-pantalla.fxml

```
<AnchorPane xmlns="http://javafx.com/javafx"
    xmlns:fx="http://javafx.com/fxml"
    fx:controller="org.example.demo.SegundaPantallaController"
    prefHeight="200.0" prefWidth="200.0">

    <Button text="Volver a pantalla de inicio"
        textAlignment="CENTER"
        onAction="#irAPantallaHello" />

</AnchorPane>
```

HelloController (con navegación)

```
public class HelloController {

    @FXML private Label welcomeText;
    @FXML private Button contador;
    @FXML private Label contando;

    @FXML
    public void initialize() {
        Contador cont = new Contador();
        contador.setOnAction(e -> {
            cont.contar();
            contando.setText(Integer.toString(cont.getContador()));
        });
    }

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("Welcome to JavaFX Application!");
    }

    @FXML
    public void irAPantalla2() throws IOException {
        HelloApplication.setRoot("segunda-pantalla");
    }
}
```

SegundaPantallaController (con navegación)

```
public class SegundaPantallaController {

    @FXML
    public void irAPantallaHello() throws IOException {
        HelloApplication.setRoot("hello-view");
    }
}
```

8.1.5 — Manipular información desde formularios

Se añaden `TextField` a la segunda pantalla para capturar datos y crear objetos.

FXML segunda-pantalla.fxml (con formulario)

```
<VBox alignment="CENTER" ... fx:controller="org.example.demo.SegundaPantallaController">
    <padding><Insets .../></padding>

    <TextField fx:id="nombreTextField" promptText="Inserta un nombre..." />
    <TextField fx:id="edadTextField" promptText="Inserta tu edad..." />
    <Button text="Guardar" onAction="#guardarPersona" />
    <Button onAction="#irAPantallaHello" text="Volver a pantalla de inicio" />
</VBox>
```

- `fx:id` → nombre para referenciar el componente en el controlador.
- `promptText` → texto placeholder cuando el campo está vacío.

Clase Persona (Modelo)

```
public class Persona {

    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }
}
```

SegundaPantallaController (con formulario)

```
public class SegundaPantallaController {

    @FXML private TextField nombreTextField;
    @FXML private TextField edadTextField;

    @FXML
    private void guardarPersona() {
        String nombre = nombreTextField.getText();
        int edad;

        try {
            edad = Integer.parseInt(edadTextField.getText());
        } catch (NumberFormatException e) {
            System.out.println("Edad inválida");
            return;
        }

        Persona persona = new Persona(nombre, edad);
        System.out.println("Persona creada: " + persona.getNombre() + ", " +
persona.getEdad());

        nombreTextField.clear();
        edadTextField.clear();
    }

    @FXML
    public void irAPantallaHello() throws IOException {
        HelloApplication.setRoot("hello-view");
    }
}
```

- `getText()` → obtiene el texto actual del `TextField`.
 - `Integer.parseInt()` + `try/catch` → convierte texto a entero de forma segura.
 - `clear()` → vacía el campo tras guardar correctamente.
-

8.1.6 — Imprimir contenido en tablas (TableView)

Se añade una TableView para mostrar en pantalla las personas creadas.

FXML segunda-pantalla.fxml (con TableView)

```
<VBox alignment="CENTER" ... fx:controller="org.example.demo.SegundaPantallaController">
    ...
    <TableView fx:id="tablaPersonas">
        <columns>
            <TableColumn fx:id="columnaNombre" text="Nombre" prefWidth="120"/>
            <TableColumn fx:id="columnaEdad" text="Edad" prefWidth="80"/>
        </columns>
    </TableView>

    <TextField fx:id="nombreTextField" promptText="Inserta un nombre..." />
    <TextField fx:id="edadTextField" promptText="Inserta tu edad..." />
    <Button onAction="#guardarPersona" text="Guardar" />
    <Button onAction="#irAPantallaHello" text="Volver a pantalla de inicio" />
</VBox>
```

SegundaPantallaController (con TableView — versión básica)

```
public class SegundaPantallaController {

    @FXML private TextField nombreTextField;
    @FXML private TextField edadTextField;
    @FXML private TableView<Persona> tablaPersonas;
    @FXML private TableColumn<Persona, String> columnaNombre;
    @FXML private TableColumn<Persona, Integer> columnaEdad;

    private ObservableList<Persona> listaPersonas = FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        columnaNombre.setCellValueFactory(data ->
            new SimpleStringProperty(data.getValue().getNombre())
        );
        columnaEdad.setCellValueFactory(data ->
            new SimpleIntegerProperty(data.getValue().getEdad()).asObject()
        );
        tablaPersonas.setItems(listaPersonas);
    }

    @FXML
    private void guardarPersona() {
        String nombre = nombreTextField.getText();
        int edad;
        try {
            edad = Integer.parseInt(edadTextField.getText());
        } catch (NumberFormatException e) { return; }
        listaPersonas.add(new Persona(nombre, edad));
        nombreTextField.clear();
        edadTextField.clear();
    }
}
```

- `ObservableList<T>` → lista reactiva: al modificarse, el `TableView` se actualiza automáticamente.
- `FXCollections.observableArrayList()` → crea una `ObservableList` vacía.
- `setCellValueFactory(...)` → indica a cada columna cómo extraer su valor del objeto de la fila.
- `SimpleStringProperty` / `SimpleIntegerProperty` → envuelven valores en propiedades JavaFX observables.
- `.asObject()` → necesario porque `TableColumn<Persona, Integer>` requiere tipo objeto, no primitivo.

Problema: datos perdidos al cambiar de pantalla

El controlador **se reinicia** al navegar fuera y volver, creando una lista vacía nueva. **Causa:** `listaPersonas = FXCollections.observableArrayList()` se ejecuta en cada instanciación del controlador.

Solución: Repositorio estático

Clase estática que actúa de almacén global durante toda la ejecución:

```
public class RepositorioPersonas {  
  
    private static final ObservableList<Persona> personas =  
        FXCollections.observableArrayList();  
  
    public static ObservableList<Persona> getPersonas() {  
        return personas;  
    }  
  
    public static void insertarPersona(Persona persona) {  
        personas.add(persona);  
    }  
}
```

- `static final` → la lista existe una sola vez en toda la JVM.
- No depende del ciclo de vida del controlador.

SegundaPantallaController (versión final con RepositorioPersonas)

```
public class SegundaPantallaController {

    @FXML private TextField nombreTextField;
    @FXML private TextField edadTextField;
    @FXML private TableView<Persona> tablaPersonas;
    @FXML private TableColumn<Persona, String> columnaNombre;
    @FXML private TableColumn<Persona, Integer> columnaEdad;

    @FXML
    public void initialize() {
        columnaNombre.setCellValueFactory(data ->
            new SimpleStringProperty(data.getValue().getNombre())
        );
        columnaEdad.setCellValueFactory(data ->
            new SimpleIntegerProperty(data.getValue().getEdad()).asObject()
        );
        tablaPersonas.setItems(RepositorioPersonas.getPersonas());
    }

    @FXML
    private void guardarPersona() {
        String nombre = nombreTextField.getText();
        int edad;
        try {
            edad = Integer.parseInt(edadTextField.getText());
        } catch (NumberFormatException e) { return; }
        RepositorioPersonas.insertarPersona(new Persona(nombre, edad));
        nombreTextField.clear();
        edadTextField.clear();
    }

    @FXML
    public void irAPantallaHello() throws IOException {
        HelloApplication.setRoot("hello-view");
    }
}
```

Resumen de clases y sus responsabilidades

Clase / Archivo	Tipo	Responsabilidad
HelloApplication.java	App principal	Inicializa la ventana, gestiona cambio de escena
HelloController.java	Controlador	Eventos pantalla 1 (contador, navegación)
SegundaPantallaController.java	Controlador	Eventos pantalla 2 (formulario, tabla, navegación)
Contador.java	Modelo	Encapsula la lógica del contador
Persona.java	Modelo	Datos de una persona (nombre, edad)
RepositorioPersonas.java	Repositorio	Lista estática y persistente de personas
hello-view.fxml	Vista	Interfaz pantalla 1
segunda-pantalla.fxml	Vista	Interfaz pantalla 2

Referencia rápida — Anotaciones y métodos

Elemento	Descripción
@FXML	Vincula campo/método Java con el componente FXML de igual fx:id
initialize()	Se ejecuta al cargar el FXML; configura eventos y valores iniciales
setText(String)	Establece el texto de una Label o Button
getText()	Obtiene el texto de un TextField
clear()	Borra el contenido de un TextField
setOnAction(e -> ...)	Registra un evento de clic en un Button desde el controlador
ObservableList<T>	Lista reactiva; el TableView se actualiza automáticamente
setCellValueFactory(...)	Indica a una TableColumn cómo obtener su valor del modelo
setItems(lista)	Asocia una ObservableList con un TableView
scene.setRoot(node)	Cambia el contenido de la ventana sin crear una nueva Stage